

PRACTICAL SESSION 6

Task 1: Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Terminal 1:

```
[(base) saiabburi@Geyasree-MacBook-Air ~ % mkdir Practical6
[(base) saiabburi@Geyasree-MacBook-Air ~ % cd Practical6
[(base) saiabburi@Geyasree-MacBook-Air Practical6 % nano server.c
```

UW PICO 5.09

File: server.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define CLIENT_FIFO "/tmp/client_fifo"

int main() {
    char message[100];
    char response[100];

    // Create named pipes
    mkfifo(SERVER_FIFO, 0666);
    mkfifo(CLIENT_FIFO, 0666);

    printf("Server started...\n");
    printf("Waiting for client message...\n");

    while (1) {
        // Open server FIFO for reading
        int server_fd = open(SERVER_FIFO, O_RDONLY);

        if (server_fd == -1) {
            perror("Error opening server FIFO");
            exit(1);
        }

        // Read message from client
        ssize_t bytes = read(server_fd, message, sizeof(message) - 1);
        close(server_fd);

        if (bytes > 0) {
            message[bytes] = '\0';

            printf("Client: %s\n", message);

            if (strcmp(message, "exit") == 0) {
                printf("Server shutting down...\n");
                break;
            }
        }
    }
}
```

```

// Read message from client
ssize_t bytes = read(server_fd, message, sizeof(message) - 1);
close(server_fd);

if (bytes > 0) {
    message[bytes] = '\0';

    printf("Client: %s\n", message);

    if (strcmp(message, "exit") == 0) {
        printf("Server shutting down...\n");
        break;
    }

    // Prepare response
    snprintf(response, sizeof(response),
             "Server received: %s", message);

    // Open client FIFO for writing
    int client_fd = open(CLIENT_FIFO, O_WRONLY);

    if (client_fd == -1) {
        perror("Error opening client FIFO");
        exit(1);
    }

    write(client_fd, response, strlen(response) + 1);
    close(client_fd);

    printf("Response sent to client.\n");
}

}

unlink(SERVER_FIFO);
unlink(CLIENT_FIFO);

return 0;
}

```

```

(base) saiabburi@Geyasree-MacBook-Air Practical6 % nano client.c
(base) saiabburi@Geyasree-MacBook-Air Practical6 % gcc server.c -o server
(base) saiabburi@Geyasree-MacBook-Air Practical6 % ls
client.c      server      server.c
(base) saiabburi@Geyasree-MacBook-Air Practical6 % gcc client.c -o client
(base) saiabburi@Geyasree-MacBook-Air Practical6 % ls
client      client.c      server      server.c
(base) saiabburi@Geyasree-MacBook-Air Practical6 % ./server
Server started...
Waiting for client message...
Client: Hello Server
Response sent to client.
Client: How is it going?
Response sent to client.
Client: exit
Server shutting down...

```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define SERVER_FIFO "/tmp/server_fifo"
#define CLIENT_FIFO "/tmp/client_fifo"

int main() {
    char message[100];
    char response[100];

    while (1) {
        printf("Enter message: ");
        fgets(message, sizeof(message), stdin);

        // Remove newline
        message[strcspn(message, "\n")] = '\0';

        // Open server FIFO for writing
        int server_fd = open(SERVER_FIFO, O_WRONLY);

        if (server_fd == -1) {
            perror("Error opening server FIFO");
            exit(1);
        }

        // Send message to server
        write(server_fd, message, strlen(message) + 1);
        close(server_fd);

        if (strcmp(message, "exit") == 0) {
            break;
        }

        // Open client FIFO for reading
        int client_fd = open(CLIENT_FIFO, O_RDONLY);

        if (client_fd == -1) {
            perror("Error opening client FIFO");
            exit(1);
        }

        // Receive response
        ssize_t bytes = read(client_fd, response, sizeof(response) - 1);
        close(client_fd);

        if (bytes > 0) {
            response[bytes] = '\0';
            printf("%s\n", response);
        }
    }

    return 0;
}
```

Task 2 : Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
[saiabburi@Geyasree-MacBook-Air Practical6] % nano signals.c  
[saiabburi@Geyasree-MacBook-Air Practical6] % gcc signals.c -o signals  
[saiabburi@Geyasree-MacBook-Air Practical6] % ./signals  
Signal handling program started.  
Process ID (PID): 4256  
Waiting for signals...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
  
SIGUSR1 received  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
  
SIGTERM received  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
^C  
SIGINT received (Ctrl+C)  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
Program is running...  
zsh: killed ./signals
```

Terminal 2:

```
[(base) saiabburi@Geyasree-MacBook-Air Practical6 % kill -SIGUSR1 78421
kill: kill 78421 failed: no such process
[(base) saiabburi@Geyasree-MacBook-Air Practical6 % kill -SIGUSR1 4256
[(base) saiabburi@Geyasree-MacBook-Air Practical6 % kill -SIGTERM 4256
[(base) saiabburi@Geyasree-MacBook-Air Practical6 % kill -9 4256
(base) saiabburi@Geyasree-MacBook-Air Practical6 %
```

```
[(base) saiabburi@Geyasree-MacBook-Air ~ % ./client
zsh: no such file or directory: ./client
[(base) saiabburi@Geyasree-MacBook-Air ~ % cd Practical6
[(base) saiabburi@Geyasree-MacBook-Air Practical6 % ls
client          client.c        server          server.c
[(base) saiabburi@Geyasree-MacBook-Air Practical6 % ./client
Enter message: Hello Server
Server received: Hello Server
Enter message: How is it going?
Server received: How is it going?
Enter message: exit
```